Technologies for IoT Ecosystem – A.A. 2025/26

Hardware Laboratory

Part 3

Communication through REST and MQTT interfaces

3.1) Arduino as an HTTP server

Create a sketch that runs an HTTP server on the Arduino and responds to GET requests from the local network. The sketch must meet the following specifications:

- The HTTP server must be implemented using the `WiFiServer` class contained in the `WiFiNINA` library.
- The server must expose one LED and the temperature sensor as "resources." In particular:
  - GET requests to the resources `http://<hostname>:<port>/led/1` or `http://<hostname>:<port>/led/0` must be handled by turning the LED on or off, respectively. The response must provide the state of the resource after the change.
  - GET requests to `http://<hostname>:<port>/temperature` must return the temperature value from the sensor in the response.
- All other requests must be handled by returning an appropriate HTTP error code.
- The body of all responses produced by the Arduino must be in JSON format and follow the SenML convention, as in the following example:

```
{
  “bn”: “ArduinoGroupX”,
  “e”: [
     {
     “n”: <”temperature”>/<”led”>,
     “t”: <timestamp using millis()>,
     “v”: value,
     “u”: “Cel”/null
     }
  ]
}
```

Data serialisation into JSON may be performed “manually” by appropriately concatenating strings.

Verify that the sketch works using a browser or a command-line utility to issue HTTP requests, such as `curl`.

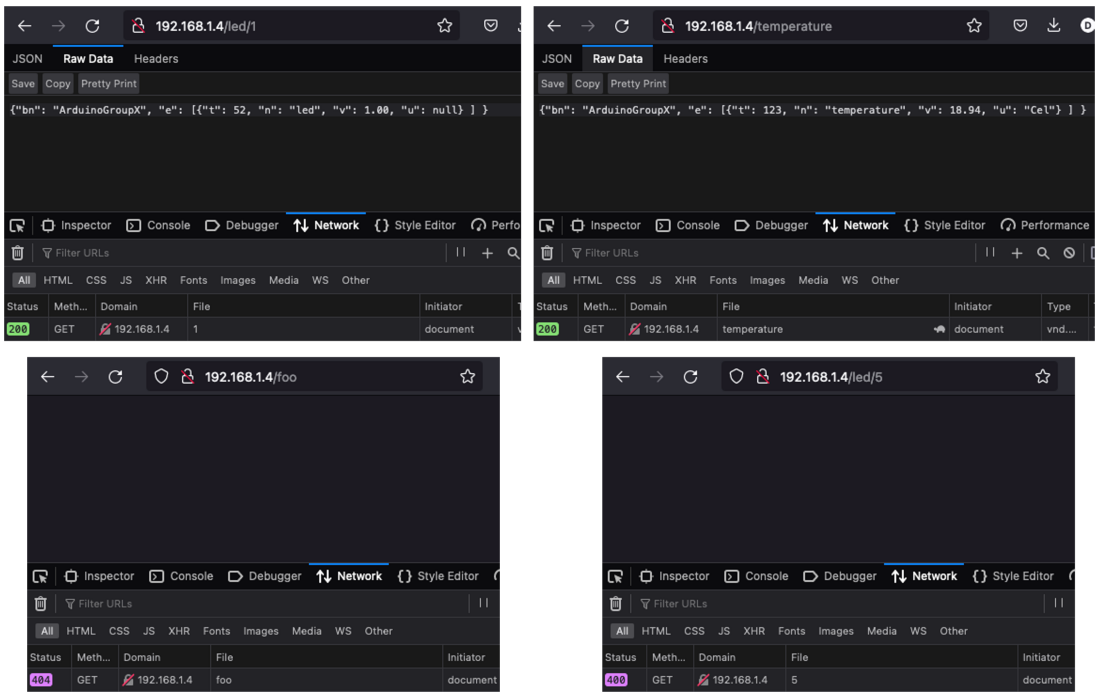

Figure 1: examples of output produced by the Arduino in response to different GET requests for Exercise 3.1.

3.2) Arduino as an HTTP client

Create a sketch that sends HTTP POST requests from the Arduino to a server to periodically log temperature sensor data. The sketch must meet the following specifications:

- The HTTP client must be implemented using the `ArduinoHttpClient` library, which in turn uses a `WiFiClient` object defined in the `WiFiNINA` library.
- The sketch must periodically read the temperature from the sensor and send a POST request to a server using the following URI: `http://<hostname>:<port>/log`
- The body of the POST request must contain the temperature data, formatted in SenML in a way analogous to Exercise 3.1.
- The sketch must print the server's HTTP response codes to the serial port.

Then extend Exercise 4 of the Software lab (Part 1) to handle requests from the Arduino. The server must meet the following specifications:

1. POST requests from the Arduino to the `/log` resource must be handled by storing the JSON in the request body in a list.
2. GET requests to the same resource must be handled by returning the entire list as a single JSON object.

Verify that the sketch works using a browser.

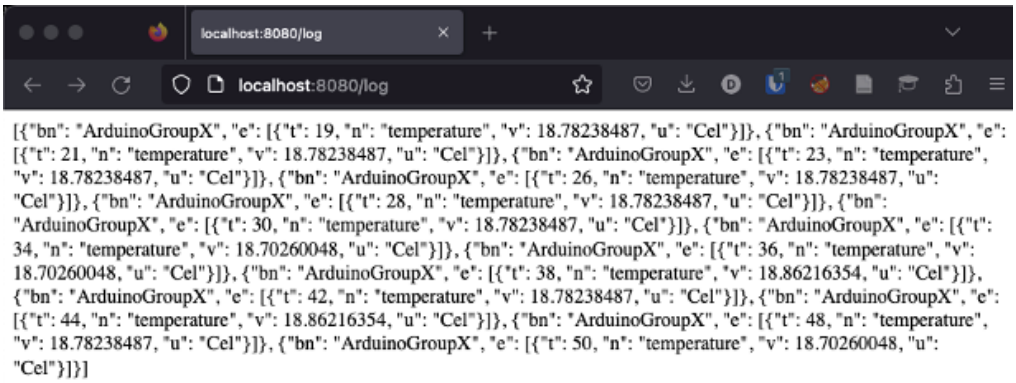

Figure 2: Example of output produced by the CherryPy server in Exercise 3.2.

3.3) Arduino as an MQTT publisher and subscriber

Create a sketch that allows the Arduino to communicate via MQTT, acting both as a publisher and as a subscriber. The sketch must meet the following specifications:

- The publish and subscribe functions must be implemented using the `PubSubClient` library, which in turn uses a `WiFiClient` object defined in the `WiFiNINA` library.
- The hostname of the MQTT broker to be used for this exercise is `test.mosquitto.org`, i.e., a public broker available for testing with small amounts of data.
- The sketch must periodically publish (e.g. every 10s) a temperature value on the topic `/tiot/group<ID>/temperature`, formatted in SenML in a way analogous to Exercise 3.1.
- The sketch must also subscribe to the topic `/tiot/group<ID>/led`, on which messages in SenML format will be sent from a PC to control one of the board's LEDs. The code must react to such messages by checking their format and, if it is correct, turning the LED on or off.

Verify that the sketch works using the command-line utilities from the Mosquitto suite on a PC (`mosquitto_sub` and `mosquitto_pub`).

```
→  lab_3.2 mosquitto_sub -h test.mosquitto.org -t '/tiot/0/temperature'
{"bn":"ArduinoGroup0","e":[{"t":81,"n":"temperature","v":18.30358887,"u":"Cel"}]}
{"bn":"ArduinoGroup0","e":[{"t":86,"n":"temperature","v":18.30358887,"u":"Cel"}]}
{"bn":"ArduinoGroup0","e":[{"t":91,"n":"temperature","v":18.22376442,"u":"Cel"}]}
{"bn":"ArduinoGroup0","e":[{"t":96,"n":"temperature","v":18.46321487,"u":"Cel"}]}
{"bn":"ArduinoGroup0","e":[{"t":101,"n":"temperature","v":18.46321487,"u":"Cel"}]}
{"bn":"ArduinoGroup0","e":[{"t":106,"n":"temperature","v":18.30358887,"u":"Cel"}]}
{"bn":"ArduinoGroup0","e":[{"t":111,"n":"temperature","v":18.30358887,"u":"Cel"}]}
{"bn":"ArduinoGroup0","e":[{"t":116,"n":"temperature","v":18.46321487,"u":"Cel"}]}
{"bn":"ArduinoGroup0","e":[{"t":121,"n":"temperature","v":18.30358887,"u":"Cel"}]}
```

Figure 3: example of data received by subscribing to the topic corresponding to group 0's temperature sensor.

```
~ mosquitto_pub -h test.mosquitto.org -t '/tiot/0/led' -m '{"bn": "ArduinoGroup0", "e": [{"n": "led", "t": null, "v": 1, "u": null}]}'
~ mosquitto_pub -h test.mosquitto.org -t '/tiot/0/led' -m '{"bn": "ArduinoGroup0", "e": [{"n": "led", "t": null, "v": 0, "u": null}]}'
```

Figure 4: examples of commands that can be sent from a PC to the Arduino of group 0 to turn the LED on and off.